

ГОСТ Р 34.11-94

Материал из Википедии — свободной энциклопедии

ГОСТ Р 34.11-94 «*Информационная технология. Криптографическая защита информации. Функция хэширования*» — устаревший российский криптографический стандарт вычисления хеш-функции, основанный на ГОСТ Р 34.10-94. В странах СНГ переиздан и используется как межгосударственный стандарт *ГОСТ 34.311-95*.

Стандарт определяет алгоритм и процедуру вычисления хеш-функции для последовательности символов. Этот стандарт является обязательным для применения в качестве алгоритма хеширования в государственных организациях РФ и ряде коммерческих организаций.

До 2013 г. ЦБ РФ требовал использовать ГОСТ Р 34.11-94 для электронной подписи предоставляемых ему документов^[3].

С 1 января 2013 года заменён РФ на ГОСТ Р 34.11-2012 «Стрибог»^[4], а с 1 июня 2019 года в странах СНГ на ГОСТ 34.11-2018^[5].

ГОСТ Р 34.11-94	
Создан	1994
Опубликован	23 мая 1994
Дата отмены	1 января 2013 года ^{[1][2]}
Преемник	ГОСТ Р 34.11-2012 «Стрибог», ДСТУ 7564:2014 «Кирупа»
Размер хеша	256 бит
Число раундов	1
Тип	хеш-функция

Содержание

Вводные обозначения

Описание

Особенности ГОСТ Р 34.11-94

Алгоритм вычисления шаговой функции хеширования

- Генерация ключей
- Шифрующее преобразование
- Перемешивающее преобразование

Узлы замены (S-блоки)

- Узлы замены, определённые документом RFC 4357
 - Идентификатор: id-GostR3411-94-TestParamSet
 - Идентификатор: id-GostR3411-94-CryptoProParamSet

Формат вывода

Примеры

- Подробный пример из стандарта
- Второй пример из стандарта
- Другие примеры
- ГОСТ хеш с «тестовым» набором параметров
- Набор параметров CryptoPro

Оценка криптостойкости

Использование

Примечания

Ссылки

Вводные обозначения

Для описания алгоритма хеширования будем использовать следующие обозначения:

- $f0g^j$ — блок длиной j бит, заполненный нулями.
- jMj — длина блока M в битах по модулю 2^{256} .
- k — слияние (конкатенация) двух блоков в один.
- $+$ — сложение двух блоков длиной 256 бит по модулю 2^{256}
- $\oplus$ — побитное сложение (XOR) двух блоков одинаковой длины.

Далее будем считать, что младший (нулевой) бит в блоке находится справа, старший — слева.

Описание

Основой описываемой хеш-функции является шаговая функция хеширования $H_{out} = f(H_{in}, m)$ где H_{out} , H_{in} , m — блоки длины 256 бит.

Входное сообщение M разделяется на блоки $m_n, m_{n-1}, m_{n-2}, \dots, m_1$ по 256 бит. В случае если размер последнего блока m_n меньше 256 бит, то к нему приписываются слева нули для достижения заданной длины блока.

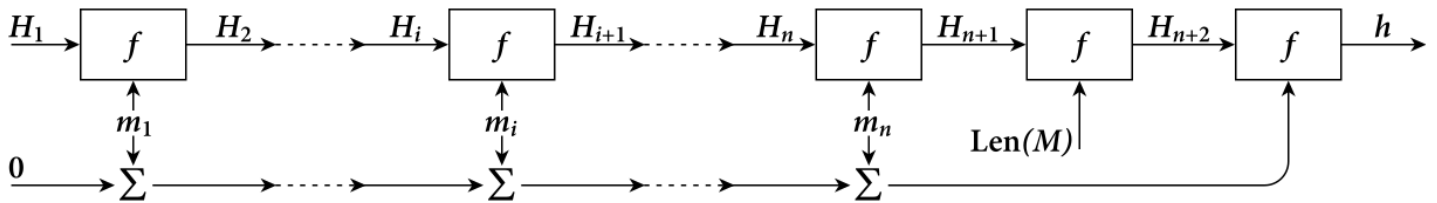
Каждый блок сообщения, начиная с первого, подаётся на шаговую функцию для вычисления промежуточного значения хеш-функции:
 $H_{i+1} = f(H_i, m_i)$

Значение H_1 можно выбрать произвольным.

После вычисления H_{n+1} конечное значение хеш-функции получают следующим образом:

- $H_{n+2} = f(H_{n+1}, L)$, где L — Длина сообщения M в битах по модулю 2^{256}
- $h = f(H_{n+2}, K)$, где K — Контрольная сумма сообщения M : $m_1 + m_2 + m_3 + \dots + m_n$

h — значение хеш-функции сообщения M



Алгоритм

1. Инициализация:

1. $h := H_1$ — Начальное значение хеш-функции. То есть — 256 битовый IV вектор, определяется пользователем.
2. $\Sigma := 0$ — Контрольная сумма
3. $L := 0$ — Длина сообщения

2. Функция сжатия внутренних итераций: для $i = 1 \dots n - 1$ выполняем следующее (пока $|M| > 256$):

1. $h := f(h, m_i)$ — итерация метода последовательного хеширования
2. $L := L + 256$ — итерация вычисления длины сообщения
3. $\Sigma := \Sigma + m_i$ — итерация вычисления контрольной суммы

3. Функция сжатия финальной итерации:

1. $L := L + j m_n j$ — вычисление полной длины сообщения
2. $m_n := f(0g^{256-jm_nj} km_n)$ — набивка последнего блока
3. $\Sigma := \Sigma + m_n$ — вычисление контрольной суммы сообщения
4. $h := f(h, m_n)$
5. $h := f(h, L)$ — MD — усиление
6. $h := f(h, \Sigma)$

4. Выход. Значением хеш-функции является h ,

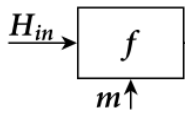
Замечание: так как длина сообщения участвует в хешировании, то нет необходимости указывать в передаваемом сообщении количество добавленных нулей к блоку m_n .

Особенности ГОСТ Р 34.11-94

- При обработке блоков используются преобразования по алгоритму ГОСТ 28147—89;
- Обрабатывается блок длиной 256 бит, и выходное значение тоже имеет длину 256 бит.
- Определяет контрольную сумму, рассчитанную по всем блокам исходного сообщения, которая является частью финального вычисления хеша, что несколько затрудняет коллизионную атаку.
- Применены меры борьбы против поиска коллизий, основанном на неполноте последнего блока.
- Обработка блоков происходит по алгоритму шифрования ГОСТ 28147—89, который содержит преобразования на S-блоках, что существенно осложняет применение метода дифференциального криптоанализа к поиску коллизий.

Алгоритм вычисления шаговой функции хеширования

Шаговая функция хеширования f отображает два блока длиной 256 бит в один блок длиной 256 бит: $H_{out} = f(H_{in}, m)$ и состоит из трех частей:



- Генерирование ключей K_1, K_2, K_3, K_4
- Шифрующее преобразование — шифрование H_{in} с использованием ключей K_1, K_2, K_3, K_4
- Перемешивающее преобразование результата шифрования

Генерация ключей

В алгоритме генерации ключей используются:

- Два преобразования блоков длины 256 бит:
 - Преобразование $A(Y) = A(y_4 \ k \ y_3 \ k \ y_2 \ k \ y_1) = (y_1 \oplus y_2) \ k \ y_4 \ k \ y_3 \ k \ y_2$, где y_1, y_2, y_3, y_4 — подблоки блока Y длины 64 бит.
 - Преобразование $P(Y) = P(y_{32} \ k y_{31} \ k \dots \ k y_1) = y_{\varphi(32)} \ k y_{\varphi(31)} \ k \dots \ k y_{\varphi(1)}$, где $\varphi(i + 1 + 4(k - 1)) = 8i + k$, $i = 0, \dots, 3$, $k = 1, \dots, 8$, а $y_{32}, y_{31}, \dots, y_1$ — подблоки блока Y длины 8 бит.
- Три константы:

$C_2 = 0$
 $C_3 = 0\text{xff}00\text{ffff}000000\text{ffff}0000\text{ff}00\text{ffff}0000\text{ff}00\text{ff}00\text{ffff}00\text{ff}00\text{ff}00\text{ff}00$
 $C_4 = 0$

Алгоритм:

$$1. U := H_{in}, \quad V := m, \quad W := U \oplus V, \quad K_1 = P(W)$$

2. Для $j = 2, 3, 4$ выполняем следующее:

$$U := A(U) \oplus C_j, \quad V := A(A(V)), \quad W := U \oplus V, \quad K_j = P(W)$$

Шифрующее преобразование

После генерирования ключей происходит шифрование H_{in} по ГОСТ 28147—89 в режиме простой замены на ключах K_i (для $i := 1, 2, 3, 4$), процедуру шифрования обозначим через E (Примечание: функция шифрования E по ГОСТ 28147 шифрует 64 битные данные 256 битным ключом). Для шифрования H_{in} разделяют на четыре блока по 64 бита: $H_{in} = h_4 \ k h_3 \ k h_2 \ k h_1$ и зашифровывают каждый из блоков:

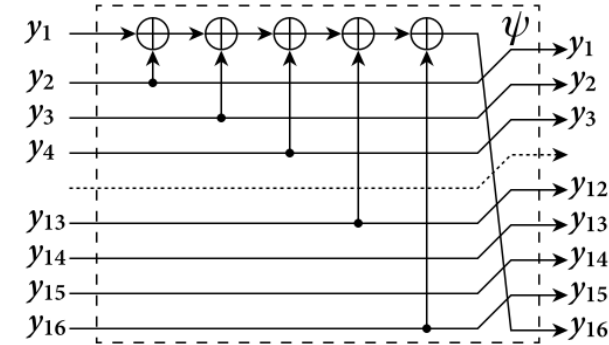
- $s_1 = E(h_1, K_1)$
- $s_2 = E(h_2, K_2)$
- $s_3 = E(h_3, K_3)$
- $s_4 = E(h_4, K_4)$

После чего блоки собирают в 256 битный блок: $S = s_4 \ k s_3 \ k s_2 \ k s_1$

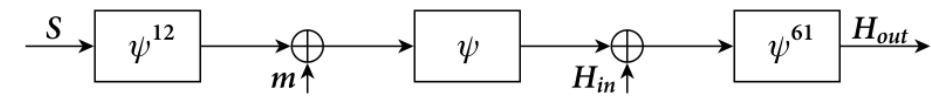
Перемешивающее преобразование

На последнем этапе происходит перемешивание H_{in} , S и m с применением регистра сдвига, в результате чего получают H_{out} .

Для описания процесса преобразования сначала необходимо определить функцию ψ , которая производит элементарное преобразование блока длиной 256 бит в блок той же длины: $\psi(Y) = \psi(y_{16} \ k y_{15} \ k \dots \ k y_2 \ k y_1) = (y_1 \oplus y_2 \oplus y_3 \oplus y_4 \oplus y_{13} \oplus y_{16}) \ k y_{16} \ k y_{15} \ k \dots \ k y_3 \ k y_2$, где $y_{16}, y_{15}, \dots, y_2, y_1$ — подблоки блока Y длины 16 бит.



Перемешивающее преобразование имеет вид $H_{out} = \psi^{61}(H_{in} \oplus \psi(m \oplus \psi^{12}(S)))$, где ψ^i означает суперпозицию $\psi \circ \psi \circ \dots \circ \psi$ длины i . Другими словами, преобразование ψ представляет собой регистр сдвига с линейной обратной связью, а индекс i указывает на количество его раундов.



Узлы замены (S-блоки)

Параметром используемого в качестве шифрующего преобразования $E(h, K)$ алгоритма ГОСТ 28147-89 является таблица из восьми узлов замены (S-блоков). ГОСТ Р 34.11-94 не фиксирует значения S-блоков и стартового вектора H_1 , что породило несовместимые реализации хеш-функции.

Широкое распространение получили два набора параметров, полагающие стартовый вектор равным нулю:

$H_1=0\times00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000,$

но имеющие значения S-блоков, указанные ниже.

Узлы замены, определённые документом RFC 4357

Идентификатор: **id-GostR3411-94-TestParamSet**

OID: 1.2.643.2.2.30.0

Номер S-блока	Значение															
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	4	A	9	2	D	8	0	E	6	B	1	C	7	F	5	3
2	E	B	4	C	6	D	F	A	2	3	8	1	0	7	5	9
3	5	8	1	D	A	3	4	2	E	F	C	7	6	0	9	B
4	7	D	A	1	0	8	9	F	E	4	6	C	B	2	5	3
5	6	C	7	1	5	F	D	8	4	A	9	E	0	3	B	2
6	4	B	A	0	7	2	1	D	3	6	8	5	9	C	F	E
7	D	B	4	1	3	F	5	9	0	A	E	7	6	8	2	C
8	1	F	D	0	5	7	A	4	9	2	3	E	6	B	8	C

Эти узлы замены определены в «приложении А» стандарта ГОСТ Р 34.11-94 для целей тестирования^[7], с рекомендацией использовать их только в проверочных примерах. Тем не менее, они получили большое распространение. Например, они описаны в RFC 5831 и их использует в своих приложениях ЦБ РФ.^[8]

Идентификатор: **id-GostR3411-94-CryptoProParamSet**

OID: 1.2.643.2.2.30.1

Номер S-блока	Значение															
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	A	4	5	6	8	1	3	7	D	C	E	0	9	2	B	F
2	5	F	4	0	2	D	B	9	1	7	6	3	C	E	A	8
3	7	F	C	E	9	4	1	0	3	B	5	2	6	A	8	D
4	4	A	7	C	0	F	2	8	E	1	6	5	D	B	9	3
5	7	6	4	B	9	C	2	A	1	8	0	E	F	D	3	5
6	7	6	2	4	D	9	F	0	A	1	5	B	8	E	C	3
7	D	E	4	1	7	0	5	A	3	C	8	F	6	2	9	B
8	1	3	A	9	5	B	4	F	8	6	7	E	D	0	2	C

Российская компания CryptoPro написала собственный «информационный» RFC 4357. Согласно ему реализации ГОСТ Р 34.11-94 должны использовать набор S-блоков, разработанный этой компанией. В известной открытой библиотеке OpenSSL начиная с версии 1.0.0 в качестве плагина появилась хеш функция ГОСТ Р 34.11-94 именно с этими параметрами. Так же данные узлы замен используются в ПО «Верба-О»^[9]

Формат вывода

Согласно ГОСТ стандарту, результатом хеш-функции является 256-битное число. Стандарт не указывает, как оно должно выводиться. Разные реализации используют различные форматы вывода, что вкупе с двумя распространёнными S-блоками усиливает путаницу.

ГОСТ Р 34.11-94 в «приложении А»^[7] оперирует с Little-endian числами. Многие реализации (в частности *rhash*, *mhash* library, консольная утилита *openssl*) выводят 32 байта результирующего хеша в шестнадцатеричном представлении, в порядке, в каком они располагаются в памяти — младшие байты первыми. Данное представление оправдывается тем, что оно используется при выводе хеш сумм таких широко распространённых алгоритмов как: MD5, SHA-1, Tiger, Whirlpool и др.

```
GOST("This is message, length=32 bytes") =
B1C466D37519B82E8319819FF32595E047A28CB6F83EFF1C6916A815A637FFFA
```

В приведённых в стандарте примерах^[7] результирующий хеш записывается как шестнадцатеричное представление 256-битного Little-endian числа. Тем самым, получается обратный порядок байт (старшие разряды первыми). Такой же порядок использует, в частности, программа *gostsum*, поставляющаяся с исходниками библиотеки OpenSSL.

```
H = FAFF37A6 15A81669 1CFF3EF8 B68CA247 E09525F3 9F811983 2EB81975 D366C4B1
```

Примеры

Подробный пример из стандарта

Вычислим хеш сообщения «This is message, length=32 bytes» с «тестовым» набором параметров.

Так как длина сообщения равна 256 битам, то нет необходимости дописывать нули. В шестнадцатеричном виде данное сообщение представляется последовательностью байт

```
54 68 69 73 20 69 73 20 6D 65 73 73 61 67 65 2C 20 6C 65 6E 67 74 68 3D 33 32 20 62 79 74 65 73
```

Эта последовательность рассматривается как Little-endian 256-битное число

```
M = 0x73657479622032333D6874676E656C202C6567617373656D2073692073696854
```

Вычисляем $H_2 = f(H_1, M)$:

■ Генерация ключей

```
K1=0x733D2C20 65686573 74746769 79676120 626E7373 20657369 326C6568 33206D54
K2=0x110C733D 0D166568 130E7474 06417967 1D00626E 161A2065 090D326C 4D393320
K3=0x80B111F3 730DF216 850013F1 C7E1F941 620C1DFF 3ABAE91A 3FA109F2 F513B239
K4=0xA0E2804E FF1B73F2 ECE27A00 E7B8C7E1 EE1D620C AC0CC5BA A804C05E A18B0AEC
```

■ Шифрующее преобразование

```
S1=0x42ABBCCE 32BC0B1B
S2=0x5203EBC8 5D9BCFFD
S3=0x8D345899 00FF0E28
S4=0xE7860419 0D2A562D
S =0xE7860419 0D2A562D 8D345899 00FF0E28 5203EBC8 5D9BCFFD 42ABBCCE 32BC0B1B
```

■ Перемешивающее преобразование

```
H2=0xCF9A8C65 505967A4 68A03B8C 42DE7624 D99C4124 883DA687 561C7DE3 3315C034
```

Вычисляем $H_3 = f(H_2, L)$:

```
L = 0x00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000100
```

■ Генерация ключей

```
K1=0xCF68D956 9AA09C1C 8C3B417D 658C24E3 50428833 59DE3D15 6776A6C1 A4248734
K2=0x8FCF68D9 809AA09C 3C8C3B41 C7658C24 BB504288 2859DE3D 666676A6 B3A42487
K3=0x4E70CF97 3C8065A0 853C8CC4 57389A8C CABB50BD E3D7A6DE D1996788 5CB35B24
K4=0x584E70CF C53C8065 48853C8C 1657389A EDCABB50 78E3D7A6 EED19867 7F5CB35B
```

■ Шифрующее преобразование

```
S =0x66B70F5E F163F461 468A9528 61D60593 E5EC8A37 3FD42279 3CD1602D DD783E86
```

■ Перемешивающее преобразование

```
H3=0x2B6EC233 C7BC89E4 2ABC2692 5FEA7285 DD3848D1 C6AC997A 24F74E2B 09A3AEF7
```

Вычисляем $H_4 = f(H_3, \Sigma)$:

```
Σ = 0x73657479622032333D6874676E656C202C6567617373656D2073692073696854
```

■ Генерация ключей

```
K1=0x5817F104 0BD45D84 B6522F27 4AF5B00B A531B57A 9C8FDPCA BB1EFCC6 D7A517A3
K2=0xE82759E0 C278D95E 15CC523C FC72EBB6 D2C73DA8 19A6CAC9 3E8440F5 C0DDB66A
K3=0x77483AD9 F7C29CAA EB06D1D7 641BCAD3 FBC3DAA0 7CB555F0 D4968080 0A9E56BC
K4=0xA1157965 2D9FBC9C 088C7CC2 46FB3DD2 7681ADCB FA4ACA06 53EFF7D7 C0748708
```

■ Шифрующее преобразование

```
S =0x2AEBFA76 A85FB57D 6F164DE9 2951A581 C31E7435 4930FD05 1F8A4942 550A582D
```

■ Применяя перемешивающее преобразование, получаем результат хеширования:

```
H4=0xFAFF37A6 15A81669 1CFF3EF8 B68CA247 E09525F3 9F811983 2EB81975 D366C4B1
```

Данное Little-endian число в машинной памяти представляется строкой байт:

```
B1 C4 66 D3 75 19 B8 2E 83 19 81 9F F3 25 95 E0 47 A2 8C B6 F8 3E FF 1C 69 16 A8 15 A6 37 FF FA
```

В записи «младшие байты первыми» имеем

```
ГОСТ("This is message, length=32 bytes") = B1C466D37519B82E8319819FF32595E047A28CB6F83EFF1C6916A815A637FFFA
```

Второй пример из стандарта

В Big-endian представлении

```
M = 0x7365747962203035203D206874676E656C20736168206567617373656D206C616E696769726F206568742065736F70707553
H = 0x0852F5623B89DD57AEB4781FE54DF14EEAFBC1350613763A0D770AA657BA1A47
```

Этот же пример в Little-endian

```
ГОСТ("Suppose the original message has length = 50 bytes") = 471ABA57A60A770D3A76130635C1FBEA4EF14DE51F78B4AE57DD893B62F55208
```

Другие примеры

Примеры^[10] в этом разделе приведены в little-endian представлении, используемом программами *mhash*, *RNHash*, *ReHash*.

ГОСТ хеш с «тестовым» набором параметров

```
ГОСТ ("") = CE85B99CC46752FFFE35CAB9A7B0278ABB4C2D2055CFF685AF4912C49490F8D
ГОСТ ("a") = D42C539E367C66E9C88A801F6649349C21871B4344C6A573F849FDCE62F314DD
ГОСТ ("abc") = F3134348C44F81B2A277729E2285EBB5CB5E0F29C975BC753B70497C06A4D51D
ГОСТ ("message digest") = AD4434ECB18F2C99B60CBE59EC3D2469582B65273F48DE72DB2FDE16A4889A4D
ГОСТ (128 символов "U") = 53A3A3ED25180CEF0C1D85A074273E551C25660A87062A52D926A9E8FE5733A4
ГОСТ (1000000 символов "a") = 5C00CCC2734CDD3332D3D4749576E3C1A7DBAF0E7EA74E9FA602413C90A129FA
```

Малейшее изменение сообщения в подавляющем большинстве случаев приводит к совершенно другому хешу вследствие лавинного эффекта. К примеру, при изменении в следующей фразе dog на cog получится:

```
ГОСТ("The quick brown fox jumps over the lazy dog") =
77B7FA410C9AC58A25F49BCA7D0468C9296529315EACA76BD1A10F376D1F4294
```

```
ГОСТ("The quick brown fox jumps over the lazy cog") =
A3EBC4DAAAB78B0BE131DAB5737A7F67E602670D543521319150D2E14EEEC445
```

Набор параметров CryptoPro

```
ГОСТ ("") = 981E5F3CA30C841487830F84FB433E13AC1101569B9C13584AC483234CD656C0
ГОСТ ("a") = E74C52DD282183BF37AF0079C9F78055715A103F17E3133CEFF1AACF2F403011
ГОСТ ("abc") = B285056DBF18D7392D7677369524DD14747459ED8143997E163B2986F92FD42C
ГОСТ ("message digest") = BC6041DD2AA401EBFA6E9886734174FEBDB4729AA972D60F549AC39B29721BA0
ГОСТ ("The quick brown fox jumps over the lazy dog") =
9004294A361A508C586FE53D1F1B02746765E71B765472786E4770D565830A76
ГОСТ ("This is message, length=32 bytes") =
2CEFC2F7B7BDC514E18EA57FA74FF357E7FA17D652C75F69CB1BE7893EDE48EB
ГОСТ ("Suppose the original message has length = 50 bytes") =
C3730C5CBCCACF915AC292676F21E8BD4EF75331D9405E5F1A61DC3130A65011
ГОСТ (128 символов "U") = 1C4AC7614691BBF427FA2316216BE8F10D92EDFD37CD1027514C1008F649C4E8
ГОСТ (1000000 символов "a") = 8693287AA62F9478F7CB312EC0866B6C4E4A0F11160441E8F4FFCD2715DD554F
```

Оценка криптостойкости

В 2008 году командой экспертов из Австрии и Польши была обнаружена техническая уязвимость, сокращающая поиск коллизий в 2^{23} раз.^{[11][12]} Количество операций, необходимое для нахождения коллизии, таким образом, составляет 2^{105} , что, однако, на данный момент практически не реализуемо. Проведение коллизионной атаки на практике имеет смысл только в случае цифровой подписи документов, причём, если взломщик может изменять неподписанный оригинал.

Использование

Функция используется при реализации систем цифровой подписи на базе асимметричного криптоалгоритма по стандарту ГОСТ Р 34.10-2001. Сообщество российских разработчиков СКЗИ согласовало используемые в Интернет параметры ГОСТ Р 34.11-94, см. RFC 4357.

- Использование в сертификатах открытых ключей.^[13]
- Использование для защиты сообщений в S/MIME (Cryptographic Message Syntax, PKCS#7).^[14]
- Использование для защиты соединений в TLS (SSL, HTTPS, WEB).^[15]
- Использование для защиты сообщений в XML Signature (XML Encryption).^[16]
- Защита целостности Интернет адресов и имён (DNSSEC).^[17]

Примечания

1. Приказ Росстандарта от 7 августа 2012 г. № 216-ст (<http://base.consultant.ru/cons/cgi/online.cgi?req=doc;base=EXP;n=539072>). Дата обращения: 31 мая 2013. Архивировано (<https://web.archive.org/web/20160304085429/http://base.consultant.ru/cons/cgi/online.cgi?req=doc;base=EXP;n=539072>) 4 марта 2016 года.
2. «... следует отметить, что отмена национального стандарта в области криптографической защиты информации не является основанием для приостановления или аннулирования действия сертификата СКЗИ, реализующего алгоритмы, определяемые данным стандартом ...» ТК26 (https://web.archive.org/web/20131207133525/https://www.tc26.ru/info/34.10-2012_34.11-2012/). Дата обращения: 31 мая 2013. Архивировано из оригинала (https://www.tc26.ru/info/34.10-2012_34.11-2012/) 7 декабря 2013 года.
3. А.В.Войлуков. ПРИКАЗ ЦБ РФ от 31.01.1995 N 02-13 "О вводе в действие в системе Центрального Банка Российской Федерации государственных стандартов Российской Федерации" (<http://docs.cntd.ru/document/9015853>) (недоступная ссылка — *история* (https://web.archive.org/web/*/http://docs.cntd.ru/document/9015853)) (31 января 1995). — ПРИКАЗ ЦБ РФ № 02-13. Дата обращения: 21 января 2019.
4. Приказ Росстандарта от 7 августа 2012 года № 216-ст (<http://docs.cntd.ru/document/902368268>). Дата обращения: 11 января 2020. Архивировано (<https://web.archive.org/web/20200111085336/http://docs.cntd.ru/document/902368268>) 11 января 2020 года.
5. Приказ Росстандарта от 4 декабря 2018 года № 1060-ст (<http://docs.cntd.ru/document/551995653>). Дата обращения: 11 января 2020. Архивировано (<https://web.archive.org/web/20200111084923/http://docs.cntd.ru/document/551995653>) 11 января 2020 года.
6. Приказ Госспецсвязи Украины от 19 ноября 2019 года № 3563/5/610 (<https://zakon.rada.gov.ua/laws/show/z1172-19>). Дата обращения: 11 января 2020. Архивировано (<https://web.archive.org/web/20200111084925/https://zakon.rada.gov.ua/laws/show/z1172-19>) 11 января 2020 года.
7. ГОСТ Р 34.11-94, ПРИЛОЖЕНИЕ А (<http://protect.gost.ru/document.aspx?control=7&id=134550>) (23 мая 1994). Дата обращения: 28 октября 2010. Архивировано (<https://www.webcitation.org/65rOcbW5y?url=http://protect.gost.ru/document.aspx?control=7>) 2 марта 2012 года.
8. Шнайер Б. 14.1 ГОСТ // Прикладная криптография. Протоколы, алгоритмы, исходные тексты на языке Си = Applied Cryptography. Protocols, Algorithms and Source Code in C. — М.: Триумф, 2002. — С. 373—377. — 816 с. — 3000 экз. — ISBN 5-89392-055-4.
9. Идентификаторы криптографических параметров алгоритмов (http://cpdn.cryptopro.ru/content/csp36/html/group__pro_cs_p_ex_CP_PARAM_OIDS.html). Дата обращения: 21 февраля 2020. Архивировано (https://web.archive.org/web/20130728082154/http://cpdn.cryptopro.ru/content/csp36/html/group__pro_cs_p_ex_CP_PARAM_OIDS.html) 28 июля 2013 года.
10. Тестовые векторы ГОСТ Р 34.11-94 (<http://gosthash.chat.ru/>) Архивная копия (<https://web.archive.org/web/20130330035751/http://gosthash.chat.ru/>) от 30 марта 2013 на Wayback Machine (рус.)
11. European cryptologists attack hash functions (<http://www.heise-online.co.uk/security/European-cryptologists-attack-hash-functions--/news/111370>) Архивная копия (<https://web.archive.org/web/20080901143646/http://www.heise-online.co.uk/security/European-cryptologists-attack-hash-functions--/news/111370>) от 1 сентября 2008 на Wayback Machine (англ.)
12. «Cryptanalysis of the GOST Hash Function», PDF (https://online.tu-graz.ac.at/tug_online/voe_main2.getVoilText?pDocumentNr=80200&pCurrPk=36649) (недоступная ссылка) Lecture Notes in Computer Science, 2008, Volume 5157/2008, 162—178, DOI: 10.1007/978-3-540-85174-5_10

13. *Leontiev, S., Ed. and D. Shefanovskij, Ed.* Using the GOST R 34.10-94, GOST R 34.10-2001 and GOST R 34.11-94 Algorithms with the Internet X.509 Public Key Infrastructure Certificate and CRL Profile (<http://tools.ietf.org/html/rfc4491>) (англ.) (май 2006). — RFC 4491. Дата обращения: 12 июня 2009. Архивировано (<https://www.webcitation.org/65dTAZWsl?url=http://tools.ietf.org/html/rfc4491>) 22 февраля 2012 года.
14. *Leontiev, S., Ed. and G. Chudov, Ed.* Using the GOST 28147-89, GOST R 34.11-94, GOST R 34.10-94, and GOST R 34.10-2001 Algorithms with Cryptographic Message Syntax (CMS) (<http://tools.ietf.org/html/rfc4490>) (англ.) (май 2006). — RFC 4490. Дата обращения: 12 июня 2009. Архивировано (<https://www.webcitation.org/61ApMs3Si?url=http://tools.ietf.org/html/rfc4490>) 24 августа 2011 года.
15. *Leontiev, S., Ed. and G. Chudov, Ed.* GOST 28147-89 Cipher Suites for Transport Layer Security (TLS) (<http://tools.ietf.org/html/draft-chudov-cryptopro-cppls>) (англ.) (декабрь 2008). — Internet-Drafts, work in progress. Дата обращения: 12 июня 2009. Архивировано (<https://www.webcitation.org/61ApNeHa4?url=http://tools.ietf.org/html/draft-chudov-cryptopro-cppls-04>) 24 августа 2011 года.
16. *S. Leontiev, P. Smirnov, A. Chelpanov.* Using GOST 28147-89, GOST R 34.10-2001, and GOST R 34.11-94 Algorithms for XML Security (<http://tools.ietf.org/html/draft-chudov-cryptopro-cpxmldsig>) (англ.) (декабрь 2008). — Internet-Drafts, work in progress. Дата обращения: 12 июня 2009. Архивировано (<https://www.webcitation.org/61ApOGMn8?url=http://tools.ietf.org/html/draft-chudov-cryptopro-cpxmldsig-07>) 24 августа 2011 года.
17. *V. Dolmatov, Ed.* Use of GOST signature algorithms in DNSKEY and RRSIG Resource Records for DNSSEC (<http://tools.ietf.org/html/draft-dolmatov-dnsext-dnssec-gost>) (англ.) (апрель 2009). — Internet-Drafts, work in progress. Дата обращения: 12 июня 2009. Архивировано (<https://www.webcitation.org/65dTB4226?url=http://tools.ietf.org/html/draft-dolmatov-dnsext-dnssec-gost-01>) 22 февраля 2012 года.

Ссылки

- Текст стандарта ГОСТ Р 34.11-94 (<http://docs.cntd.ru/document/1200004857>) «Информационная технология. Криптографическая защита информации. Функция хэширования»
- Текст стандарта ГОСТ 34.311-95 (<http://docs.cntd.ru/document/1200004857>) «Информационная технология. Криптографическая защита информации. Функция хэширования»
- ГОСТ Р 34.11 — 94. Функция хеширования. Краткий анализ. (https://web.archive.org/web/20061010004030/http://www.ssl.stu.neva.ru/psw/crypto/GOST_R_34.11-94_Pub.pdf)

Источник — https://ru.wikipedia.org/w/index.php?title=ГОСТ_Р_34.11-94&oldid=134155509

Эта страница в последний раз была отредактирована 12 ноября 2023 в 16:32.

Текст доступен по лицензии Creative Commons «С указанием авторства — С сохранением условий» (CC BY-SA); в отдельных случаях могут действовать дополнительные условия.

Wikipedia® — зарегистрированный товарный знак некоммерческой организации Фонд Викимедиа (Wikimedia Foundation, Inc.)